

Bridge Automation Framework - Technical Documentation

1. Overview

This project automates the **Bridge Course Teacher Registration Portal**, covering the end-to-end flow from basic details entry to payment gateway interaction. It is built using **Selenium WebDriver (Python)** following the **Page Object Model (POM)** design pattern.

2. Project Structure

```
Bridge-Automation.-Framework-/ ■■■■ config/ ■ ■■■■ config.ini # Centralized
configuration (URLs, Timeouts, Payment Details) ■■■■ logs/ # Automation logs ■■■■
pages/ # Page Object Classes ■ ■■■■ base_page.py # Base class with generic wrapper
methods (wait + click/type) ■ ■■■■ registration_page.py ■ ■■■■ payment_flow_page.py
■ ■■■■ ... ■■■■ tests/ ■ ■■■■ conftest.py # Pytest fixtures (setup/teardown) ■ ■■■■
test_registration.py# Main regression test script ■■■■ utilities/ ■ ■■■■
custom_logger.py # Logging utility ■ ■■■■ data_utils.py # Data generation (Email,
Files, Random Data) ■ ■■■■ read_properties.py # Config reader ■■■■ test_data/ #
Artifacts for testing ■ ■■■■ dummy.jpg # Auto-generated ■ ■■■■ dummy.pdf #
Auto-generated ■ ■■■■ email_counter.txt # Tracks unique email generation ■■■■
requirements.txt
```

3. Key Features

- **Centralized Configuration:** All timeouts, URLs, and static data are managed in `config/config.ini`.
- **Robust Interaction:** generic `do_click`, `do_send_keys`, `is_visible` methods in `BasePage` use explicit waits (`WebDriverWait`) automatically.

Data Management: `utilities/data_utils.py` handles:

- Incremental email generation (to avoid "Email already exists" errors).
- Dynamic creation of dummy PDF/JPG files for upload testing.
- Random data generation using `Faker`.

Payment Automation:

- Supports SabPaisa gateway.
- Handles iframe switches automatically.
- Configurable card details in `config.ini`.

4. Setup & Execution

Prerequisites

- Python 3.9+
- Chrome Browser

Installation

```
pip install -r requirements.txt
```

Running Tests

Run the main regression suite:

```
python3 -m pytest tests/test_registration.py -v --html=reports/report.html
```

Configuration

Edit `config/config.ini` to change:

- `base_url`: Target environment.
- `explicit_wait`: Global timeout trigger.
- `[payment]`: Update card details if test cards change.

5. Reporting

Logs are stored in `logs/automation.log`.

A generic pytest HTML report is generated if `pytest-html` is installed and used with `--html` flag.